Advanced Algorithms and Parallel Programming

PRAM

architectures · algorithms · performance evaluation

academic year 2026-2027

Where we are going

From the abstract model to concrete decisions about how many processors to use

1

Model

RAM → PRAM: what we idealize and why.

2

Memory rules

EREW, CREW, CRCW: which algorithms are legal.

3

Patterns

Map, reduce, prefix-sum, matrix multiply.

4

Performance

work, cost, speedup, efficiency.

5

Scaling

Strong vs weak scaling with measurable recipes.

Guiding idea: PRAM is not real hardware; it is a language for reasoning about the limits and patterns of parallelism.

Why start from a model?

A good model separates algorithmic ideas from machine details

Without a model

different caches
different network
different scheduler
different input

With PRAM

parallel time $T_p(n)$
work $W(n)$
memory conflicts
reusable patterns

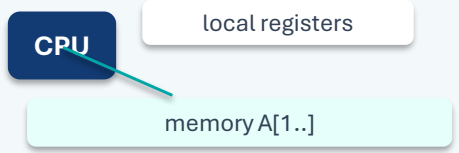
Right question: “how much of the computation can run in parallel if memory were not the bottleneck?”

RAM → PRAM: the abstraction in detail

PRAM keeps the RAM unit-cost idea, but replicates the processor and makes communication happen through shared memory.

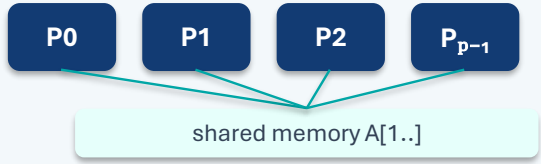
RAM: sequential model

- One processor with a program counter and registers.
- Memory is an unbounded array of cells; each cell stores one word/integer.
- Basic instructions: load/store, arithmetic, comparison, branch.
- Each instruction costs one time step.
- $T_1(n)$ = number of executed instructions; space = used memory cells.



PRAM: parallel model

- Processors P_0, P_1, …, P_{p-1} are identical; each knows its own id i.
- Each processor has private registers/local variables.
- Input X[·], output Y[·], and shared memory A[·] are globally addressable.
- All active processors execute rounds in lock-step; processors may be idle.
- Communication is only by reading/writing shared memory.



One PRAM round

1. read X or A
2. compute locally
3. write Y or A
4. barrier

The memory-conflict rule (EREW/CREW/CRCW) decides whether same-cell simultaneous reads/writes are legal.

Idealization: unit-time shared-memory access ignores cache, bandwidth, network latency, and synchronization overhead.

Key takeaway: RAM defines the sequential cost baseline T_1; PRAM defines parallel time T_p as synchronous rounds under explicit memory-access rules.

A PRAM round is a global barrier

All processors execute the same sequence of phases in lock-step

1 **read input / shared** — 2 **local compute** — 3 **write output / shared** — 4 **barrier → next round**

Example: A[i] = B[i−1] + 1

Each processor reads a neighbor and writes its own cell in the same round. We must know whether two processors read/write the same cell.

Motivation: counting rounds gives a clear measure of parallel time $T_p(n)$.

Memory conflicts: what is legal?

Read the acronym as [read rule][write rule]: CREW = Concurrent Read + Exclusive Write.

Model	Simultaneous reads	Simultaneous writes	Typical use
EREW	distinct addresses only	distinct addresses only	no same-round sharing
CREW	same address OK	distinct addresses only	broadcast x in matrix-vector
CRCW	same address OK	same address OK, with a rule	global OR, priority minimum

Acronym decoder

First letter controls reads; second letter controls writes.

C — **Concurrent**: same address allowed

E — **Exclusive**: distinct addresses only

R — **Read**: rule applies to reads

W — **Write**: rule applies to writes

EREW = Exclusive Read + Exclusive Write

CREW = Concurrent Read + Exclusive Write

CRCW = Concurrent Read + Concurrent Write

Concurrent Write (CW)

- What value gets written finally?
  - Priority CW: processors have priority based on which value is decided, the highest priority is allowed to complete WRITE
  - Common CW: all processors are allowed to complete WRITE ***iff*** all the values to be written are equal. Any algorithm for this model has to make sure that this condition is satisfied. If not, the algorithm is **illegal** and the machine state will be undefined.
  - Arbitrary/Random CW: one randomly chosen processor is allowed to complete WRITE

Metrics: “faster” is not enough

Separate time, total work, and wasted resources

$S_p(n) = T^*(n) / T_p(n)$

Speedup: how many times faster than the best sequential version.

$E_p(n) = T1(n) / p\,T_p(n)$

Efficiency: useful fraction of the available compute power.

$\text{Cost} = p \cdot T_p(n)$

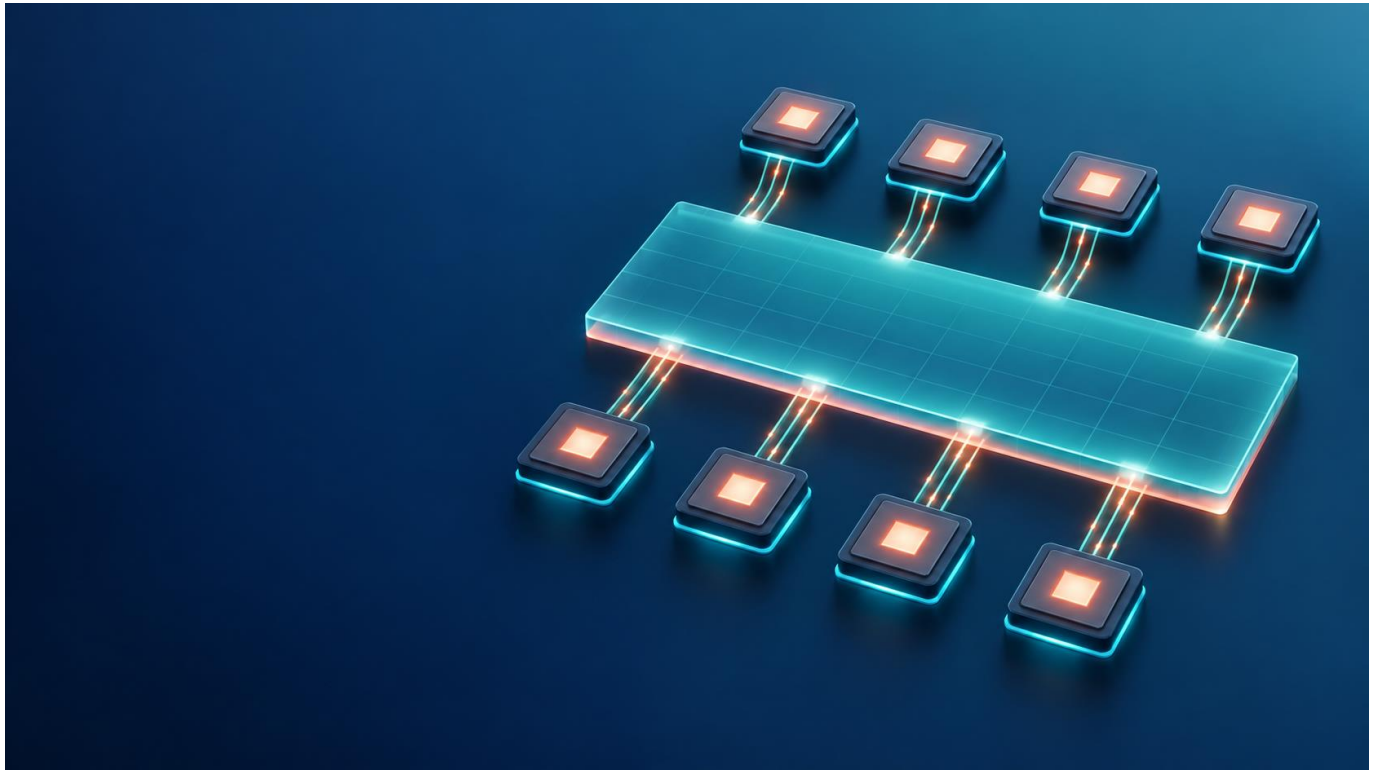
PRAM cost: allocated processors × rounds. Cost-optimal means $\Theta(T_1)$.

Rule of thumb

A parallel algorithm is convincing if it reduces T_p without exploding work or cost. Many idle processors are wasted resources.

From now on, every example should include: algorithm, memory model, T_p, speedup, efficiency.

Questions → definitions → decisions



Parallel Random Access Machine (PRAM)

Q1 What should be the baseline?

Motivating question
If a parallel code beats my first serial implementation, is the algorithm truly good?

Definition in use
$T^*(n)$: time of the best sequential algorithm for input size n.
$T_1(n)$: time of this PRAM algorithm using one processor.
Usually $T^*(n) \le T_1(n)$: a parallel algorithm may do extra bookkeeping.

Use $T^*(n)$, not a weak serial baseline.

Application: summing n values



If the one-processor version is $\Theta(n \log n)$, using T_1 as the baseline hides the extra work.

Decision: compare to the best known sequential algorithm first; then analyze the parallel implementation.

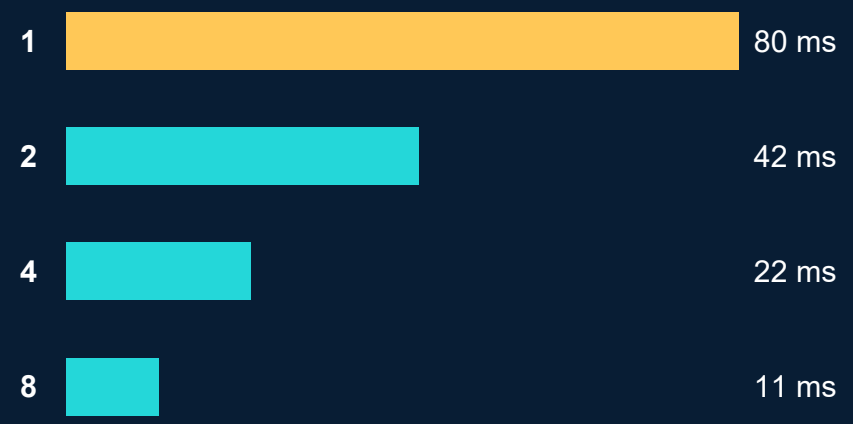
Q2 How much faster is the parallel run?

Motivating question
After adding p processors, how do I measure the time-to-solution gain?

Definition in use
$T_p(n)$: time to solve the same problem on p processors.
Speedup $S_p(n)$ compares the best sequential time with the parallel time.
Ideal strong scaling gives $S_p \approx p$.

$$S_p(n) = T^*(n) / T_p(n)$$

Application: matrix-vector multiply



With p = 8: $S_8 = 80 / 11 \approx 7.3\times$

Use speedup to answer: “How much sooner do I get the result?”

Q3 Are the processors doing useful work?

Motivating question
A run is faster, but did we waste many processors on overhead or idle time?

Definition in use
Efficiency $E_p(n)$ is speedup per processor.
When $T^*(n) \approx T_1(n)$, $E_p = S_p / p$.
$E_p \leq 1$: perfect efficiency means every processor contributes ideally.

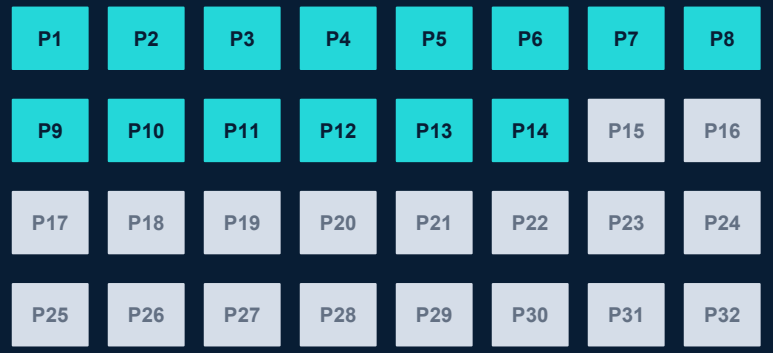
$$E_p(n) = T_1(n) / (p \cdot T_p(n))$$

Application: same code, different p

$p = 8, S_8 = 7.3 \Rightarrow E_8 \approx 91\%$



$p = 32, S_{32} = 14 \Rightarrow E_{32} \approx 44\%$



Classroom heuristic: if efficiency collapses early, inspect communication, load balance, memory bandwidth, and serial sections before adding more processors.

Q4 What is the resource cost?

Motivating question
The run is faster, but did it consume more total processor-time than the sequential solution?

Definition in use
Cost $C_p(n)$ is processors × time.
A faster run can be more expensive if it uses many processors inefficiently.
Cost is the PRAM analogue of a compute budget.

$$C_p(n) = p \cdot T_p(n)$$

Application: choose by deadline and budget

p	T_p	S_p	E_p	C_p
1	80 ms	1.0×	100%	80
8	11 ms	7.3×	91%	88
32	5.7 ms	14×	44%	182



Decision: use p = 32 only when the deadline justifies the extra cost.

Area ≈ processor-time

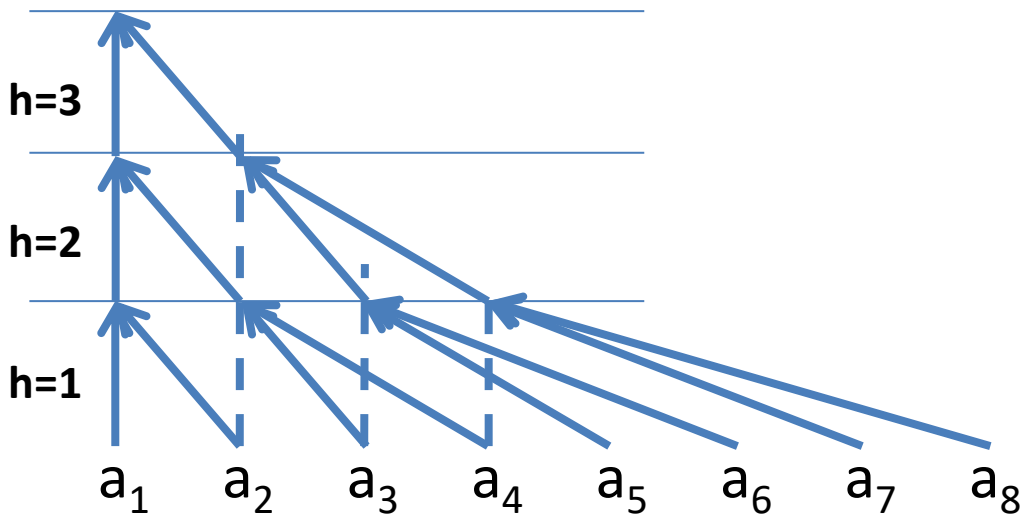
Sum of a vector

The PRAM algorithm

// Sum vector A(*)

```
Begin
        B(i) := A(i)
        For h=1:log(n)
            if i ≤ n/2^h then
                B(i) = B(2i-1) + B(2i)
End
```

// B(1) holds the sum



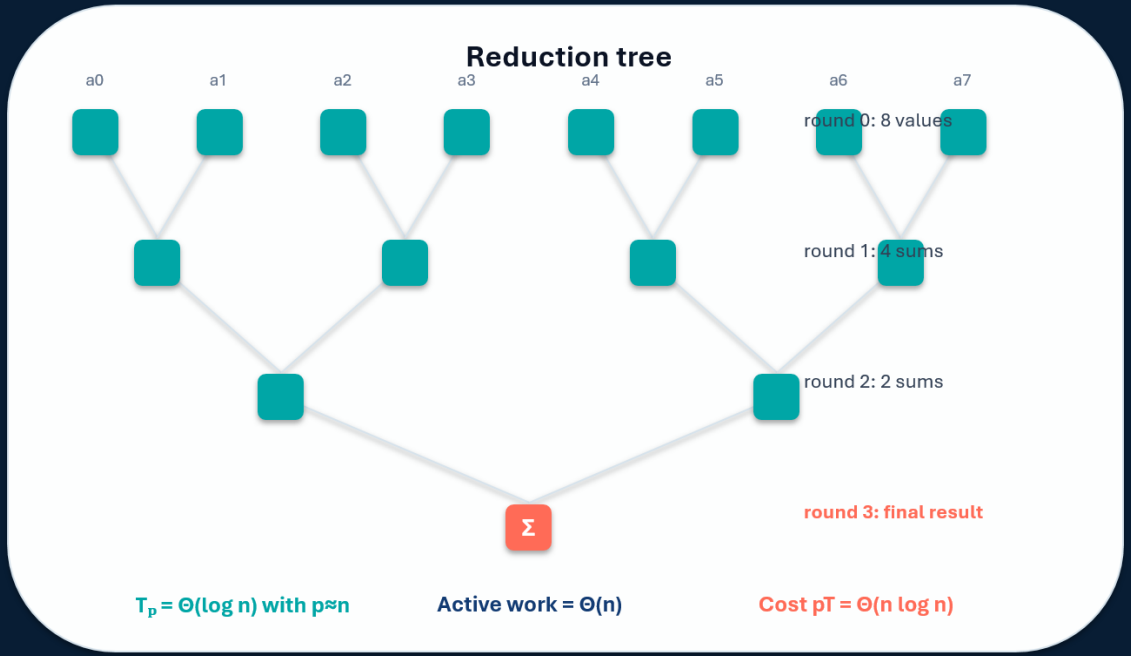
Q5 Did the parallel algorithm create extra work?

Motivating question
A PRAM schedule may reserve many processors. Which operations are actually performed?

Definition in use
Work W(n): total number of basic operations performed by all processors.
A parallel algorithm is work-efficient when W(n) = O(T*(n)).
Cost C_p counts reserved processor-time; work counts real operations.

$$W(n) \le C_p(n)$$

Application: reduction of n numbers



Useful additions: $n - 1 \Rightarrow W = \Theta(n)$

But if we reserve p = n processors for log n rounds, C = Θ(n log n). Chunk first, then reduce, to keep cost near Θ(n).

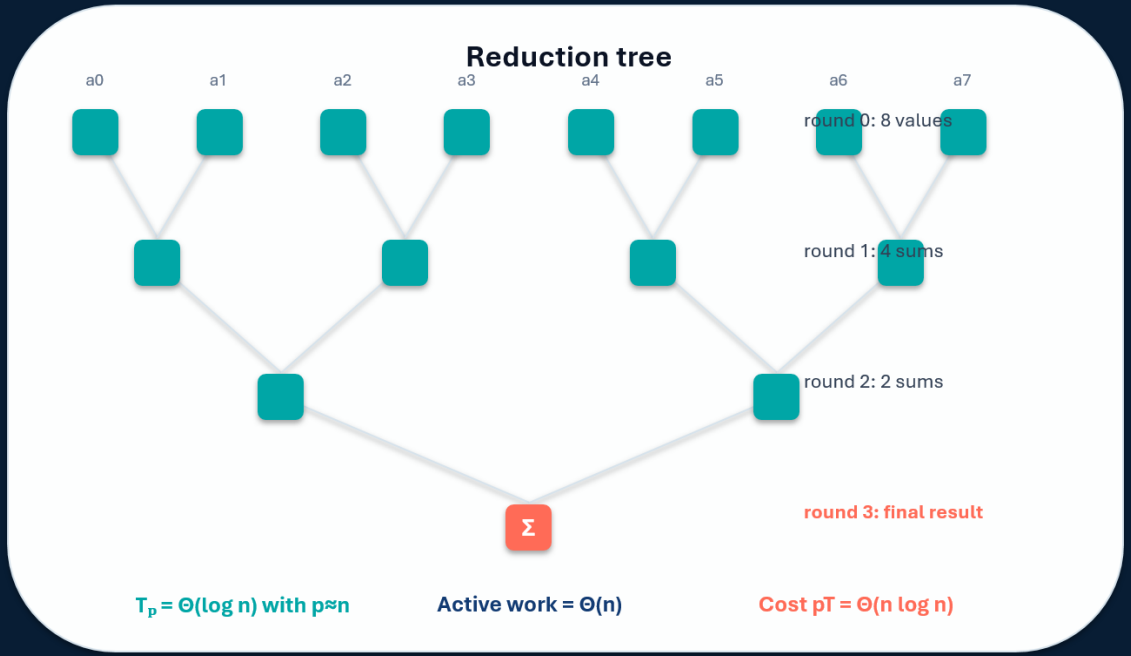
Q6 What is the best time imaginable?

Motivating question
If processors were unlimited, what part of the computation still has to happen in sequence?

Definition in use
T∞(n): shortest runtime on any number of processors.
It is the critical path: dependencies that cannot be parallelized away.
This gives an upper bound on speedup.

$$S_p(n) \le T_1(n) / T\infty(n)$$

Application: tree reduction



For $n = 2^{20}$ values:

$T_1 \approx 1{,}048{,}576$ operations
$T\infty \approx \log_2(n) = 20$ steps
maximum speedup ≈ 52,429×

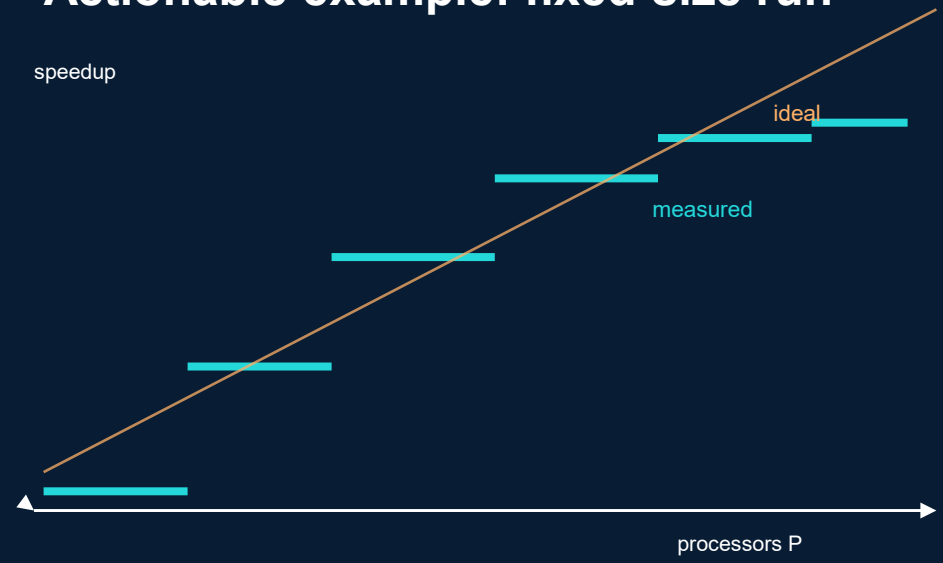
Q7 When should I stop adding processors?

Motivating question
Which definition turns scaling curves into an actionable decision?

Definition in use
Use speedup, efficiency, cost, and T∞ together.
Adding processors is useful only while time decreases enough to justify extra processor-time.
Past the critical-path limit, efficiency must fall.

Stop when ΔT is small or E_p is below target.

Actionable example: fixed-size run



p	S_p	E_p
8	7.3×	91%
16	12×	75%
32	20×	63%
64	28×	44%
128	34×	27%

Decision: if your target is $E_p \geq 50\%$, stop around p = 32 in this run; p = 64 is faster but less efficient.

Q8 How should I report PRAM performance?

Motivating question
Which quantities make a scaling result reproducible and interpretable?

Definition in use
Report the model and the experiment before the formula.
Then report T^*, T_p, S_p, E_p, C_p, W, and the T_∞ limit.
This separates algorithmic parallelism from measurement artifacts.

Minimal report = setup + timings + interpretation

A clean reporting template

Field	Example
Problem size	$n = 2^{20}$ values
PRAM model	CREW, tree reduction
Baseline	$T^*(n) = \Theta(n)$
Timing	median T_p over 5 runs
Metrics	S_p, E_p, C_p, W
Limit	$T_\infty = \Theta(\log n)$

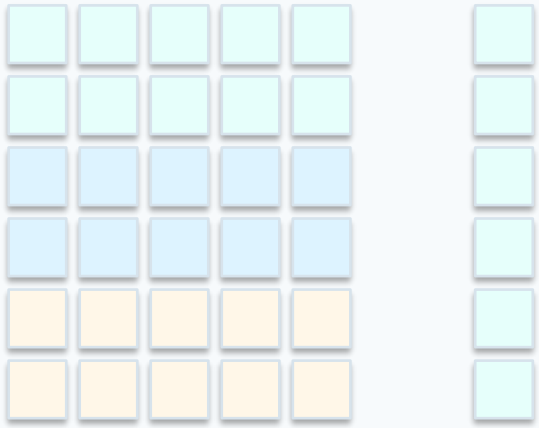
Colab exercise idea: implement a reduction, record T_p for $p \in \{1,2,4,8,16\}$, and compute S_p, E_p, and C_p in a small table.
COLAB = PRAM_definitions_QA_Colab_EN.ipynb

EXAMPLES

Pattern 1 · Map: matrix-vector multiply

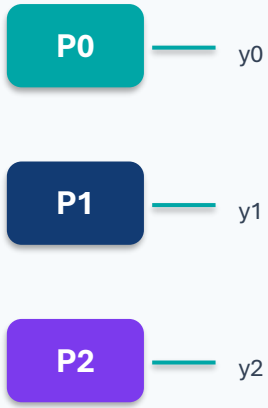
Each processor computes an independent block of rows

$$y = A\,x$$



A split by rows

x read by all



Why is it embarrassingly parallel?

Each y_i depends on one row of A and on all of x, but not on other y_j. Therefore no reductions across processors are needed.

Model: CREW

Actionable: to decompose a problem, look for independent outputs before thinking about synchronization.

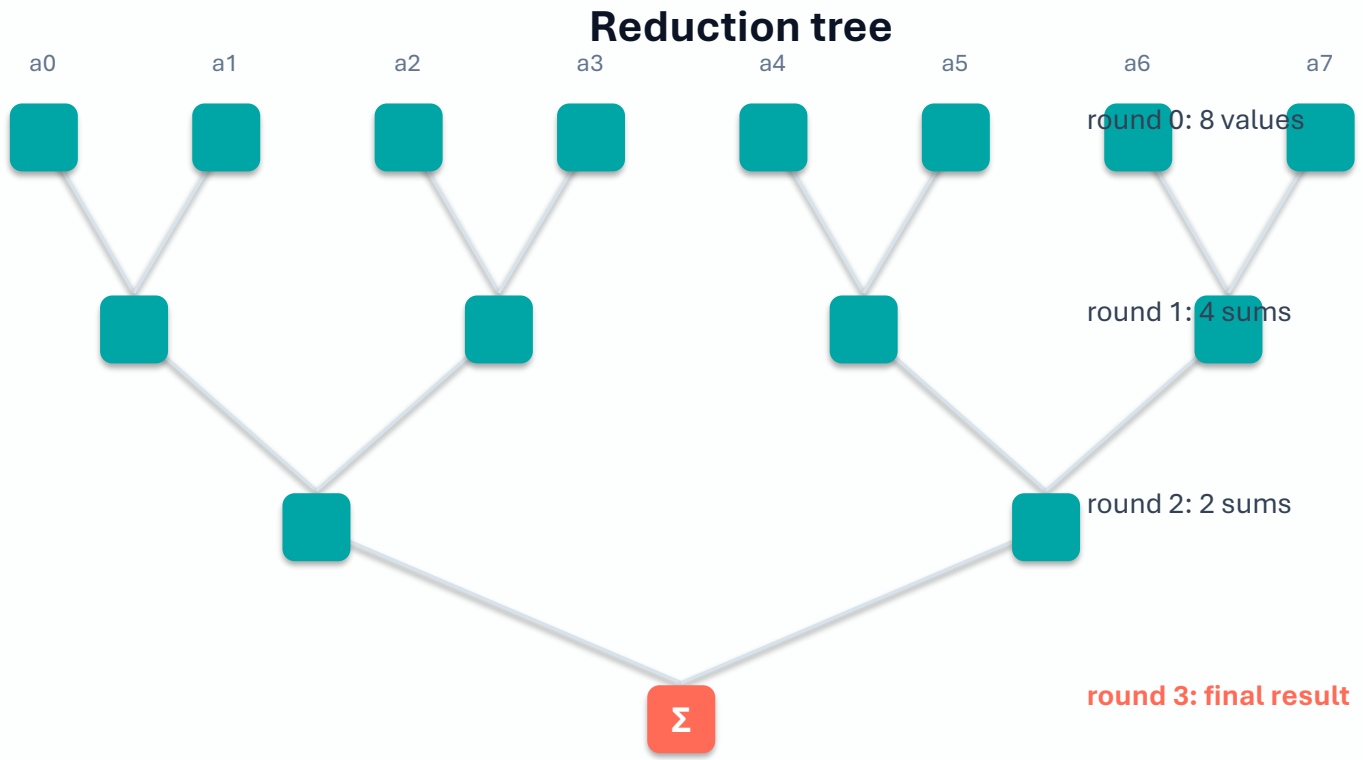
Pattern 2 · Reduce: tree sum

From n elements to 1 result in $\log_2(n)$ rounds if you have enough processors

Why reduction matters

Reduction is the primitive that turns a global dependency into a sequence of local combinations.

Examples: sum, max, OR, dot product, convergence tests.



$T_p = \Theta(\log n)$ with $p \approx n$

Active work = $\Theta(n)$

Cost $pT = \Theta(n \log n)$

Reduce when p ≪ n: chunk + tree

The practical version does not assign one processor to every element

Algorithm

1. each P_i sums n/p local elements
2. the p partial sums are reduced in a tree
3. P_0 writes the final result

$$T_p(n) = \Theta(n/p + \log p)$$

Actionable decision

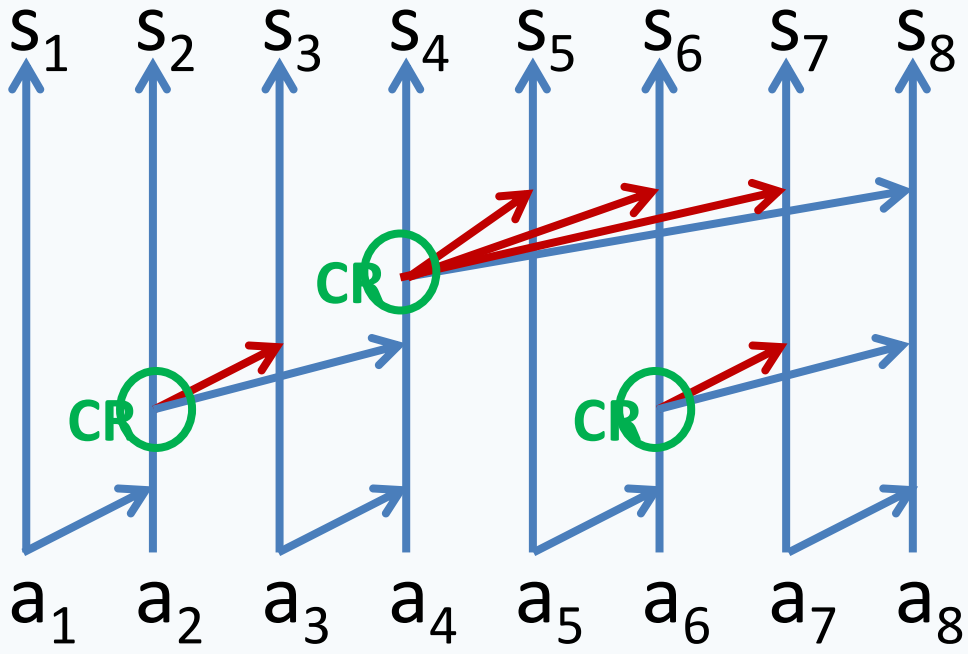
Add processors while the n/p term dominates log p. When log p becomes comparable to local work, efficiency drops.

Rule of thumb: $p \log_2 p \lesssim n$

n	safe p	why
10^6	≤ 65k	p log p stays well below n
10^4	≤ 512	visible reduction overhead beyond this
10^2	≤ 16	problem too small for much parallelism

Pattern 3 · Prefix sum

Not just a sum: compute $S_i = a_1 + \cdots + a_i$ for every i



PRAM algorithm: Depth Θ(log n).

Concrete uses

array compaction · stream compaction · partitioning for sorting · BFS · graph algorithms

PRAM is strong, and that is why it is useful

An algorithm that is slow even on PRAM is unlikely to become fast on real hardware

Natural

p processors can perform at most p operations per round.

Simple

enables clean correctness and complexity proofs.

Strong

unit-time access to all memory: ignores network/cache/synchronization.

Theoretical benchmark

if it does not scale in the ideal model, hardware alone is not enough.

Use PRAM as the first question: “is there enough parallelism?” Then use the real system to ask “how much does communication cost?”

PRAM scalability lemmas

What happens when the ideal PRAM uses more processors or memory than we actually have?

The problem

PRAM algorithms often assume an unbounded supply of processors and shared memory cells. The lemmas show that this is not magic: excess resources can be simulated, but time increases predictably.

Lemma 1 · fewer processors

If an algorithm runs on p PRAM processors for t rounds, then p' < p real processors can simulate it by batching virtual processors.

$$T' = O(t \cdot p / p')$$

same shared memory, fewer processors

Interpretation: one real processor time-slices about p/p' virtual processors in each PRAM round.

Lemma 2 · fewer memory cells

If the PRAM algorithm uses m shared cells, then m' < m cells can simulate it by partitioning memory into segments and serializing accesses.

$$T' = O(t \cdot m / m')$$

with max(p,m') processors

Interpretation: each simulated read/write may need a sweep over m/m' memory segments.

Takeaway: PRAM scalability can be reasoned about as a resource/time trade-off before implementation.

Using the lemmas: an actionable checklist

Convert an ideal PRAM cost into a back-of-the-envelope execution plan

Checklist

1. **Start from the ideal bound**
   p virtual processors, m shared cells, t PRAM rounds
2. **Map to the target machine**
   p' physical workers, m' fast/shared cells
3. **Estimate simulation overhead**
   processor factor p/p' and memory factor m/m'
4. **Decide if the model is still useful**
   keep overhead within the experiment budget

Example: simulate an ideal PRAM run

Ideal algorithm

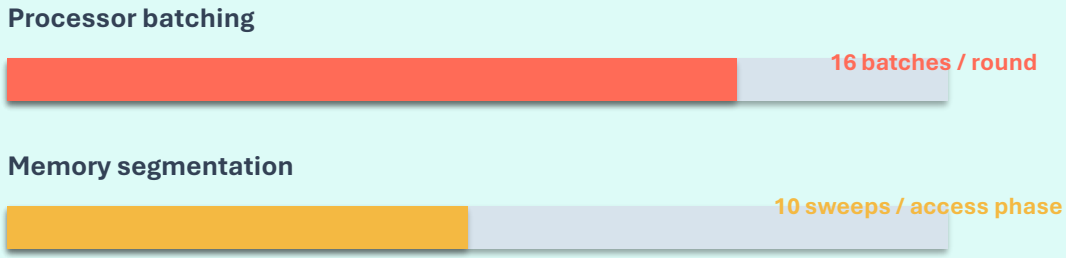
p = 1024
t = 1,000 rounds
m = 1,000,000 cells

Available machine

p' = 64 workers
m' = 100,000 cells

Estimate

processor overhead ≈ 16×
memory overhead ≈ 10×



Colab 04 lets students change p, p', m, m' and see the overhead curves.

Strong and weak scaling

Two different questions, two different experiments, two different decisions

Strong vs weak scaling: the one-line difference

Do not compare plots unless the experiment fixes the same quantity

Strong scaling

fixed N, increasing P

Question: how much do I reduce time-to-solution?

Metrics: $T_p(N)$, S_p, E_p

Guiding theory: Amdahl

Weak scaling

fixed N/P, increasing P

Question: can I solve larger problems in the same time?

Metrics: $T_p(P \cdot n_0)$, Eweak

Guiding theory: Gustafson

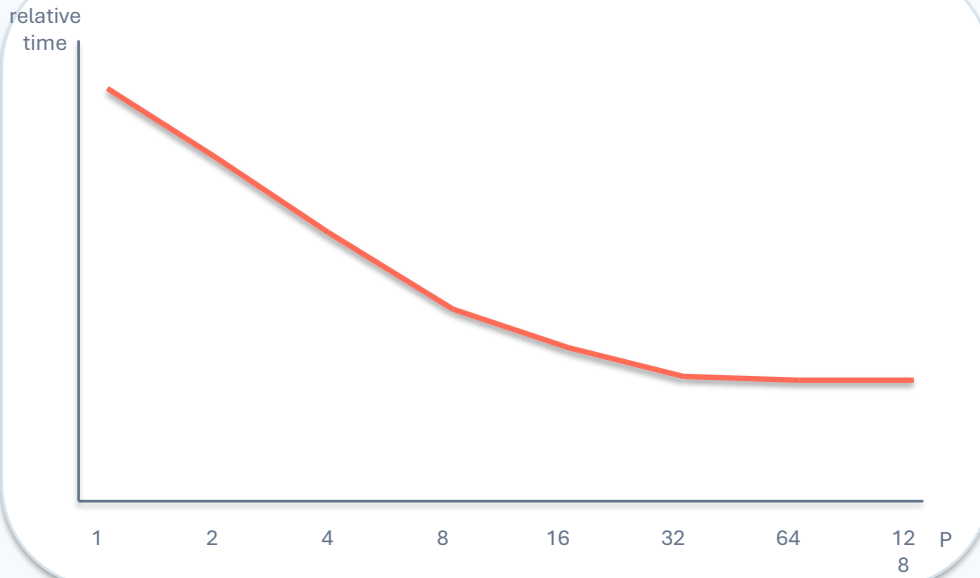
Strong = finish sooner. Weak = do more in the same time.

Strong scaling: benchmark recipe

Example: 2D stencil on a 4096×4096 grid for 100 steps, fixed N

Recipe

1. Choose N large but fixed
2. Run P = 1, 2, 4, 8, ...
3. Record median(T_p)
4. Compute $S = T_1/T_p$ and $E = S/P$
5. If $E < 60–70\%$, stop or diagnose



Reading the plot

The plateau means adding processors no longer reduces time: change the algorithm, decomposition, or communication.

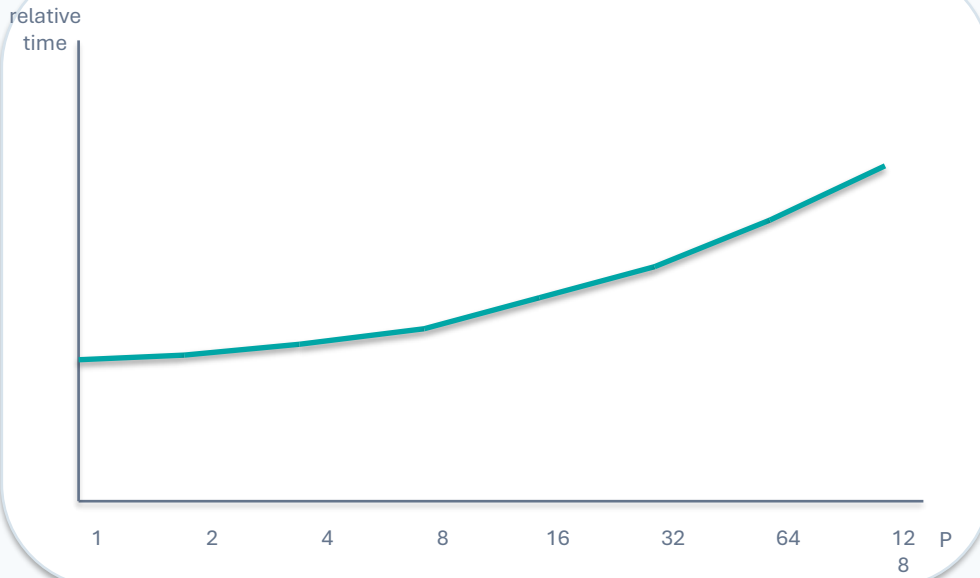
Decision: not “use all cores,” but “use cores while time decreases and efficiency remains acceptable.”

Weak scaling: benchmark recipe

Example: each processor keeps a local 1024×1024 grid

Recipe

1. Fix local work n_0
2. Set $N = P \cdot n_0$
3. Measure $T_p(P \cdot n_0)$
4. $E_{weak} = T_1(n_0)/T_p(P \cdot n_0)$
5. Diagnose growth: halo, I/O, global synchronizations



Reading the plot

A nearly flat curve is good: the system absorbs the problem growth. The slope measures communication and coordination cost.

Decision: increase P when you want more resolution, more data, or more scenarios within the same time budget.

Amdahl vs Gustafson: no contradiction

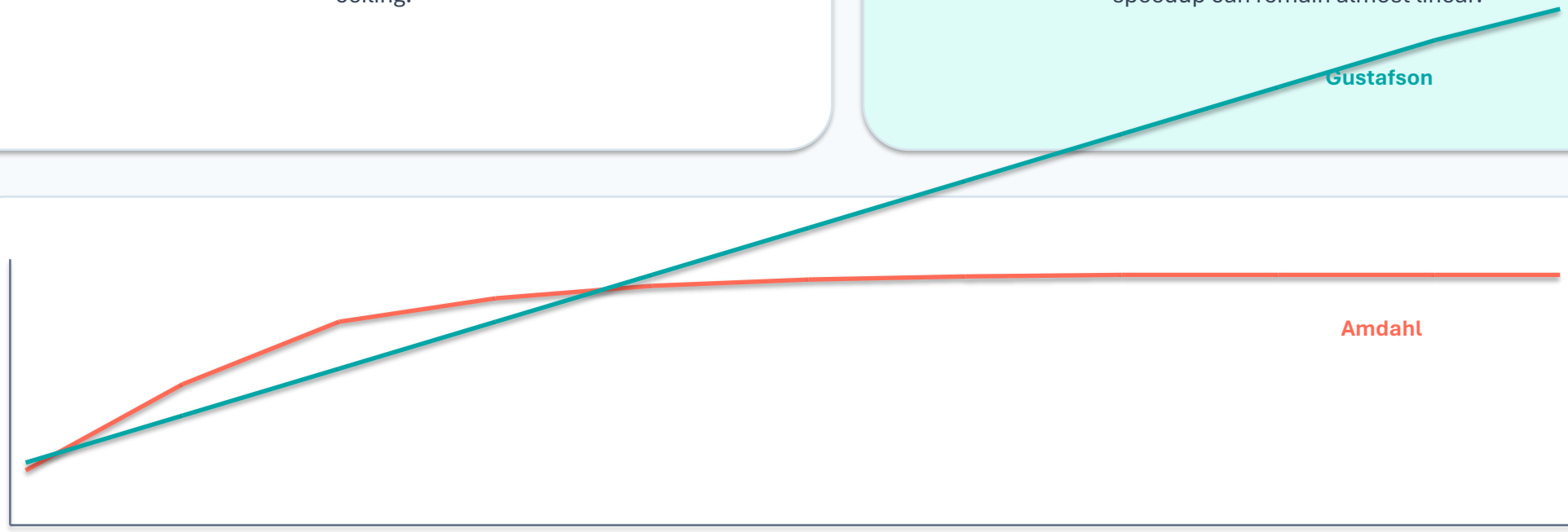
The experiment invariant changes

Amdahl: $S(P)=1/(s+(1-s)/P)$

Fixed-size / strong scaling. If the serial fraction s is nonzero, speedup has a ceiling.

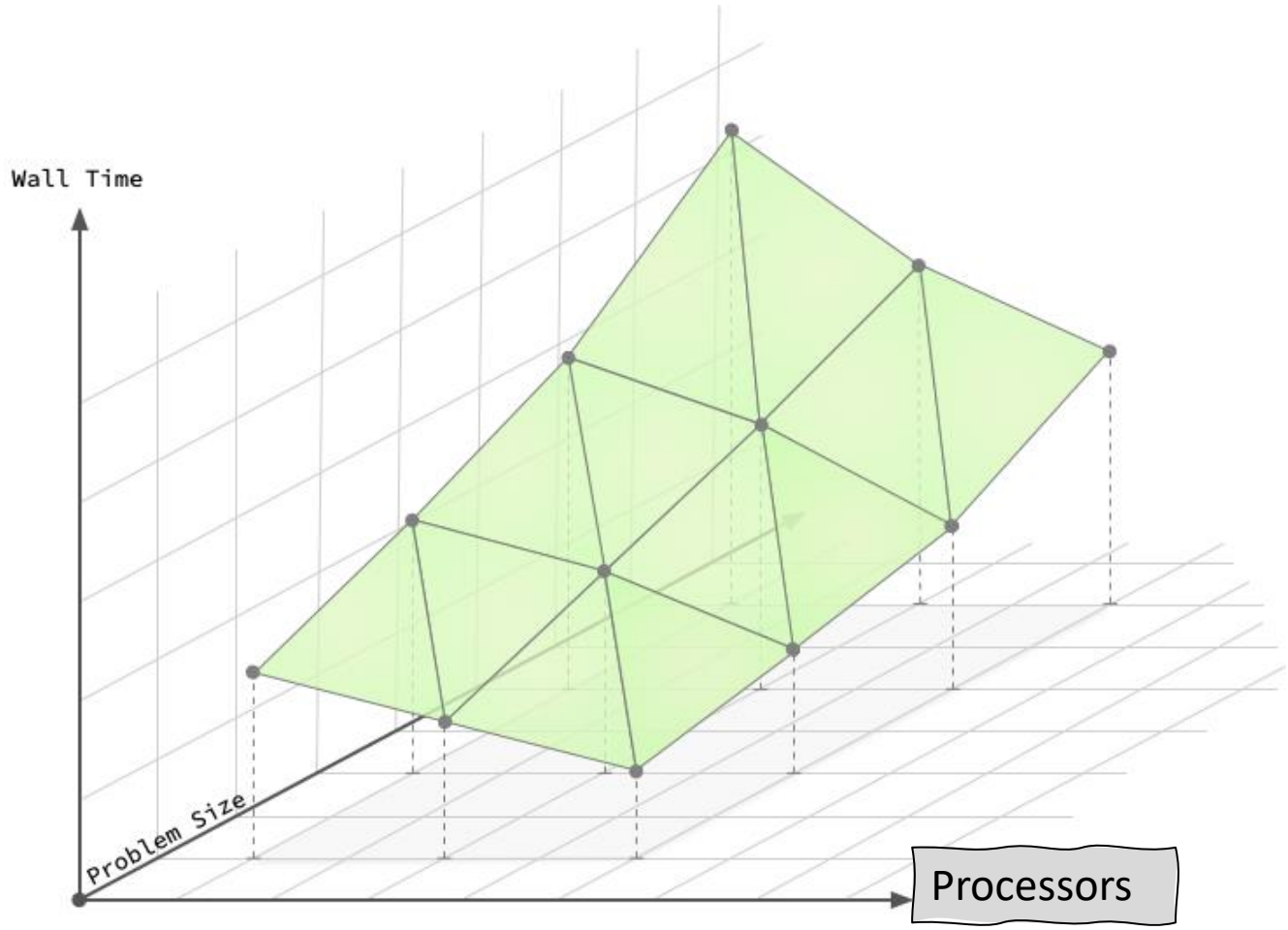
Gustafson: $S(P)=s+(1-s)P$

Fixed-time / weak scaling. The parallel part grows with P, so scaled speedup can remain almost linear.

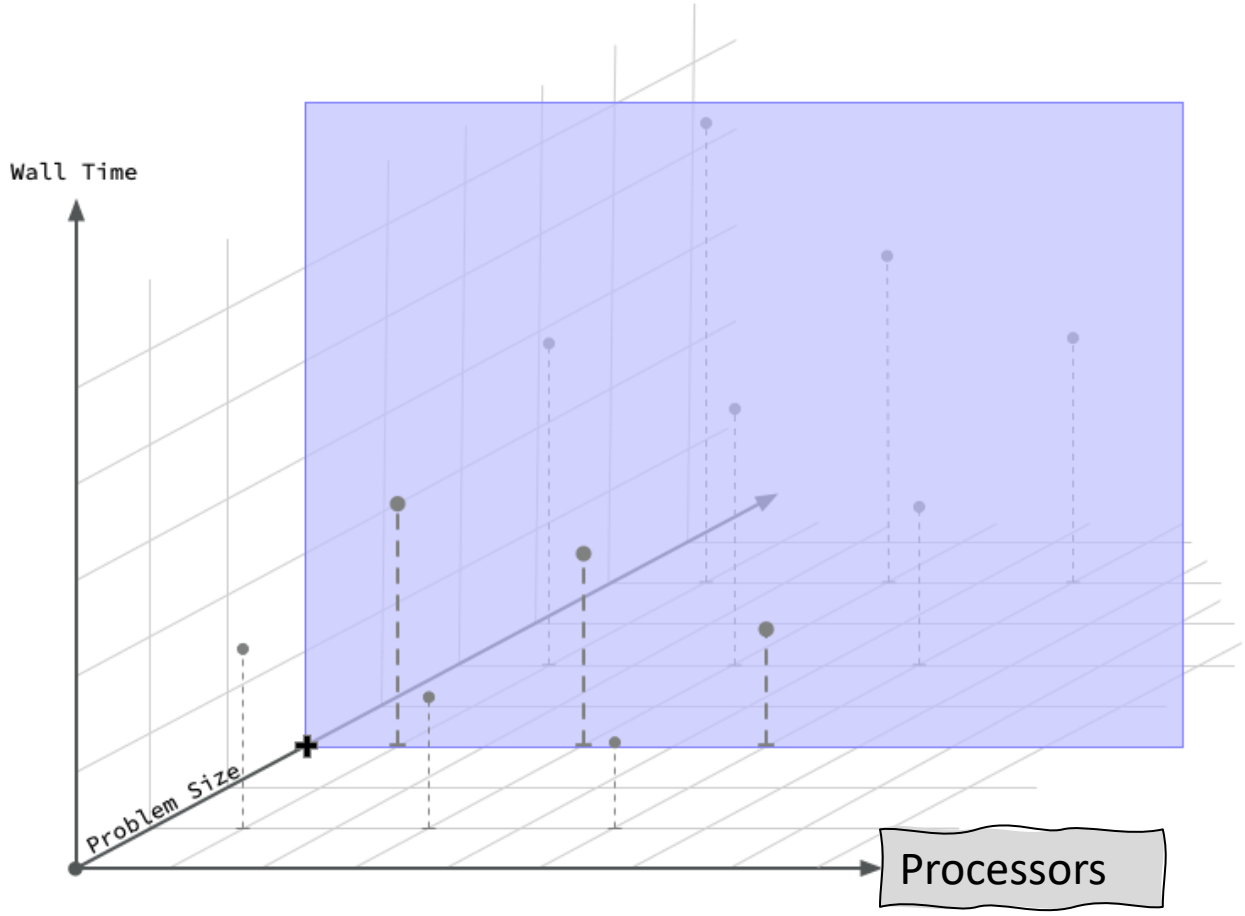


Ref.: Amdahl, 1967; Gustafson, 1988.

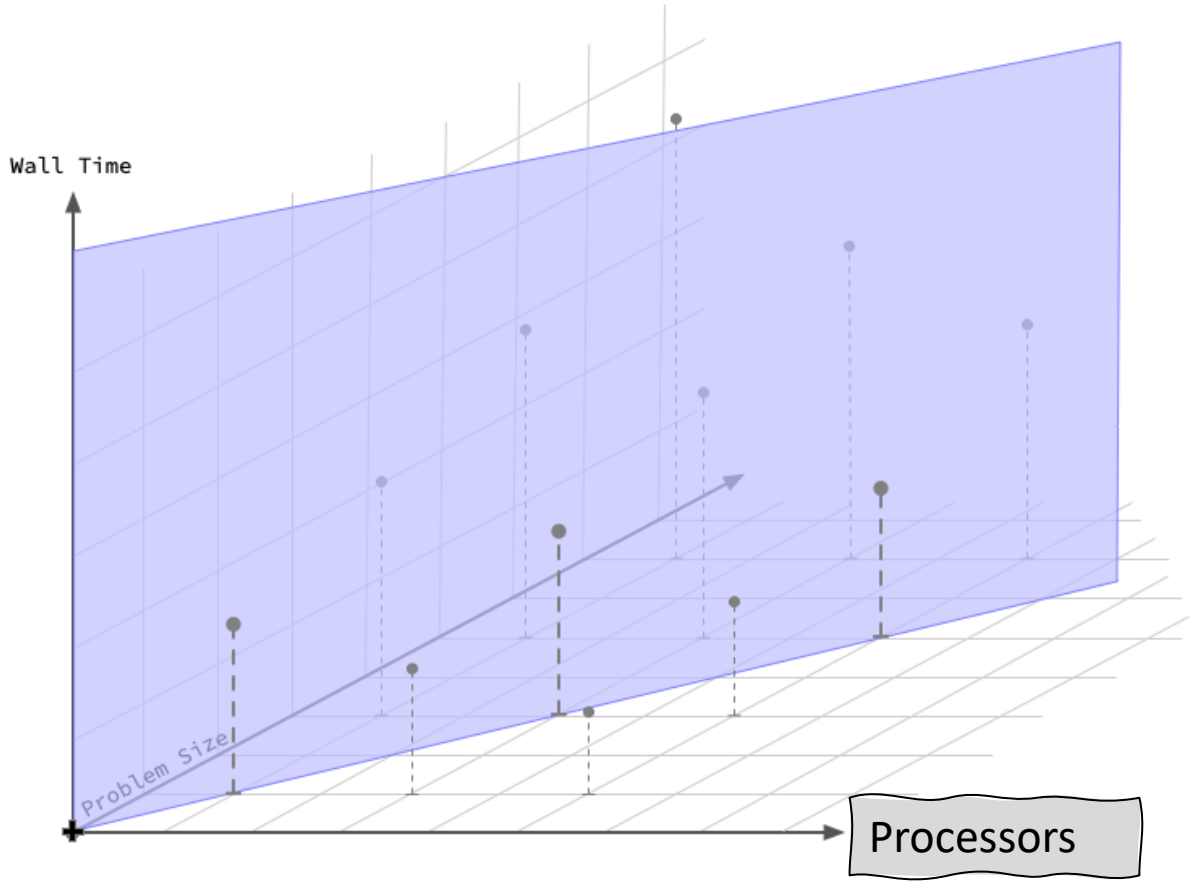
The relationship between problem size, number of processors, and time to solution can be visualized as a three-dimensional surface.



Strong scaling is concerned with a slice of the scaling surface at a particular problem size.



Weak scaling is concerned with a slice of the scaling surface where the work per task is constant.



Three mini Colab labs to make the concepts executable

The notebooks are designed to be uploaded to Google Colab and edited by students



Labs

01 · PRAM access + reduce + prefix-sum
simulate EREW/CREW/CRCW conflicts; measure T, S, E

02 · Amdahl vs Gustafson
slider for serial fraction and strong/weak comparison

03 · Stencil strong/weak scaling
halo-exchange model and benchmark decisions

Classroom use

10 min demo + 15 min guided exercise + 5 min discussion of the generated plots.

Takeaways

What students should retain

1. **PRAM is a model, not a machine**
2. **EREW/CREW/CRCW rules motivate the algorithms**
3. **Map/reduce/prefix-sum are reusable patterns**
4. **Speedup without efficiency can be waste**
5. **Strong and weak scaling answer different decisions**

Main sources

Vishkin PRAM notes · JaJa, Introduction to Parallel Algorithms · Amdahl (1967) · Gustafson (1988)